Heart Disease Risk Prediction — MLOps

Assignment 01

V Gupta (2024AC05229)

Table of Contents

1. OVERVIEW

This project implements an end-to-end Machine Learning Operations (MLops) classifier that predicts the risk of heart disease from patient health data (UCI Heart Disease; Cleveland dataset) and deploys it as a cloud-ready, monitored REST API. The solution covers the full MLOps lifecycle: data acquisition, EDA, feature engineering, model training with hyperparameter tuning, experiment tracking (MLflow), automated testing and CI/CD (GitHub Actions), containerization (Docker), production deployment (Kubernetes via Minikube), and monitoring (Prometheus + Grafana).

> **Repository link:** https://github.com/2024ac05229/Mlops
> **Deployed API:** local Kubernetes cluster (Minikube) — see Section 8 for access instructions.
> **Recording Link:** https://drive.google.com/file/d/1GXonxL_9Aj16MX2n2EhdOVv5D6ueTr1-/view?usp=sharing

2. SETUP / INSTALL INSTRUCTIONS

Full commands are documented in `README.md`. Below is a summary of the setup and run sequence:

```
python -m venv .venv
# Activate environment (Windows / Linux)
.venv\Scripts□ctivate   # OR source .venv/bin/activate
pip install -r requirements.txt

# Execution sequence
python data/download_dataset.py       # Ingest, clean, and split dataset
python notebooks/eda.py               # Generate EDA visualization plots
python src/train.py                   # Train classifiers & log runs to MLflow
python -m pytest tests/               # Run automated unit & API tests

# Docker commands
docker build -t heart-disease-api:latest .
docker run -d -p 8000:8000 --name heart-disease-api-instance heart-disease-api:latest
```

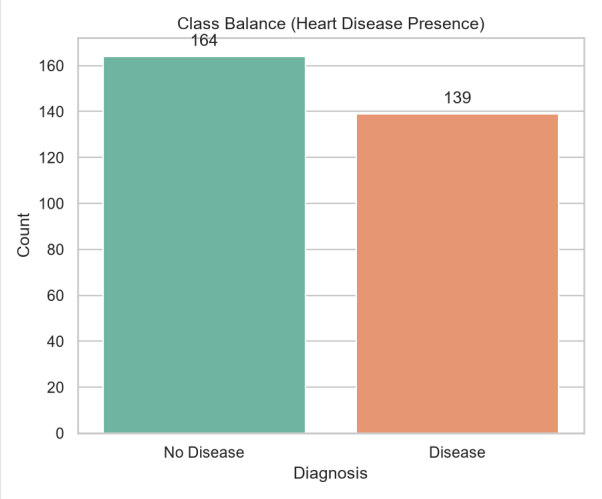
All dependencies are strictly pinned in `requirements.txt`. The complete pipeline executes from a clean virtual environment setup without requiring manual tasks or folder creation.

3. DATA ACQUISITION & EDA

Dataset: Retrieved from the UCI Machine Learning Repository, the Cleveland Heart Disease dataset consists of 303 patient records, containing 13 clinical features (age, sex, chest pain type, resting blood pressure, cholesterol, ECG results, max heart rate, exercise-induced angina, ST depression, etc.) and a target label representing presence/absence of heart disease (collapsing values 1-4 into class 1).

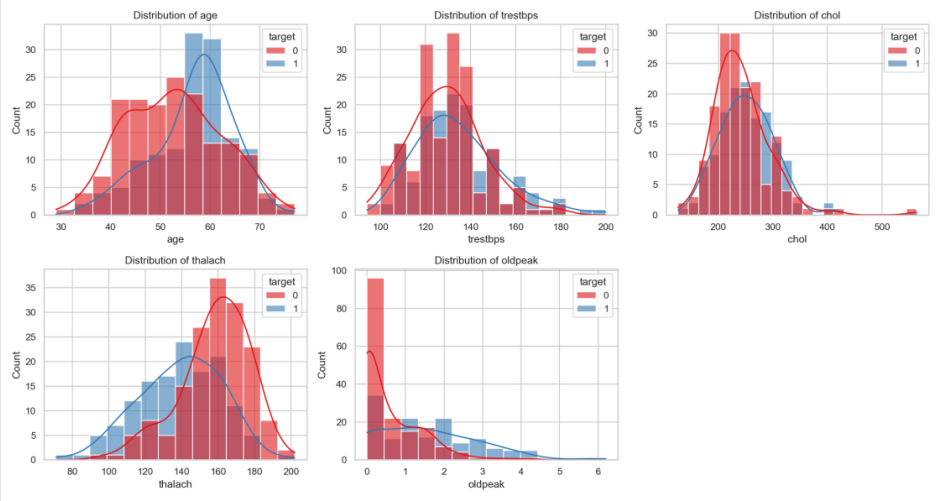
The ingestion module `data/download_dataset.py` downloads the dataset directly from the UCI archive, parses missing values (which are denoted as `?` in the raw files for columns `ca` and `thal`), handles conversions, and performs an 80-20 stratified split into training (242 rows) and testing (61 rows) CSV files under `data/processed/`.

Class balance: The dataset is reasonably balanced, containing 164 records with no disease (54.1%) and 139 records showing presence (45.9%):

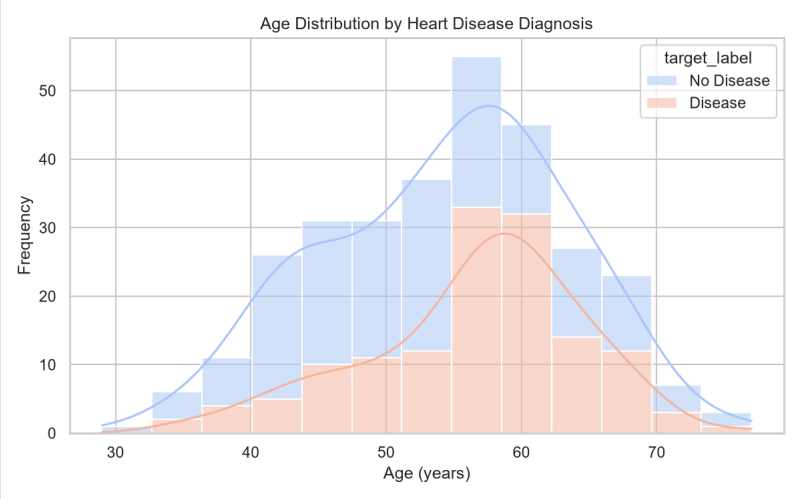


Missing Values and Distributions

Missing values: Identified missing values in columns `ca` (4 records) and `thal` (2 records) are handled dynamically during the preprocessing stage using mode imputation. Visualized metrics verify zero missing values post-cleaning:

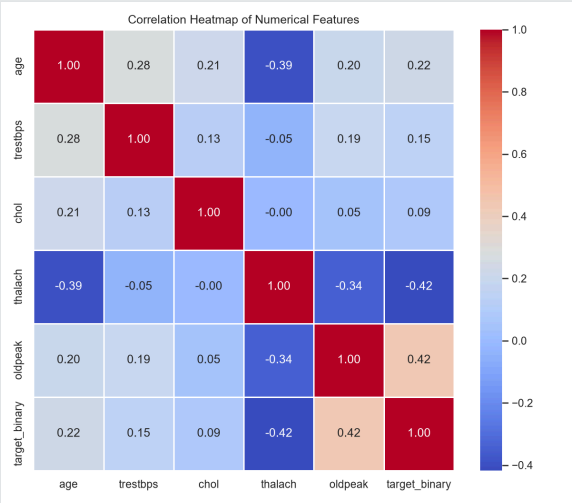


Feature distributions: Continuous and categorical variable counts are analyzed relative to the target label. For example, age distributions show a higher frequency of heart disease diagnoses in older brackets:

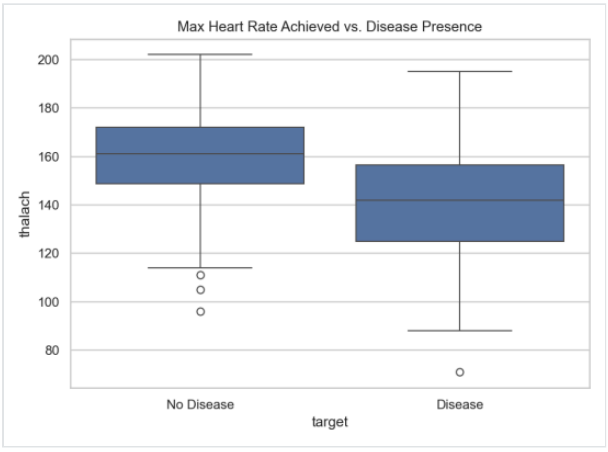


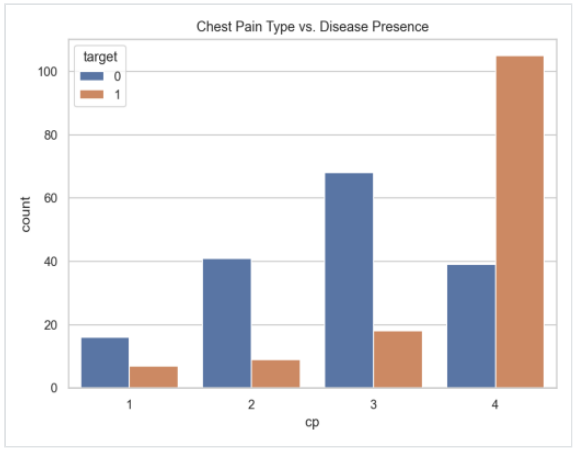
Correlation Heatmap & Boxplots

Correlation analysis: Attributes like `thal`, `ca`, `exang`, and `oldpeak` show the strongest positive correlation with the target classification, whereas `thalach` (maximum heart rate achieved) displays a strong negative correlation:



Feature relationships: Further exploratory boxplots verify that patients diagnosed with heart disease exhibit lower maximum heart rates achieved (`thalach`) and higher ST depression levels (`oldpeak`) relative to rest:





4. FEATURE ENGINEERING & MODEL DEVELOPMENT

Preprocessing Pipeline: Defined as a modular scikit-learn `ColumnTransformer` in `src/preprocessing.py`. Numeric features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`) undergo median value imputation followed by standard scaling (`StandardScaler`). Categorical features (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`) are processed using most-frequent imputation and one-hot encoding (`OneHotEncoder`). Wrapping the preprocessing pipeline and the classifier in one `Pipeline` guarantees that identical transformations are applied at training and inference time.

Models Trained: Logistic Regression and Random Forest models are trained using 5-fold stratified cross-validation on ROC-AUC scores for hyperparameter grid tuning:

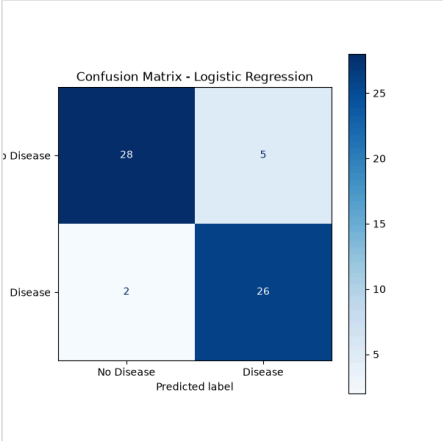
- **Logistic Regression:** Regularization strength `C` ∈ {0.01, 0.1, 1, 10}, `penalty` = l2, `solver` = lbfgs.
- **Random Forest:** Estimators count ∈ {50, 100, 200}, max depth ∈ {None, 5, 10}, min split ∈ {2, 5}.

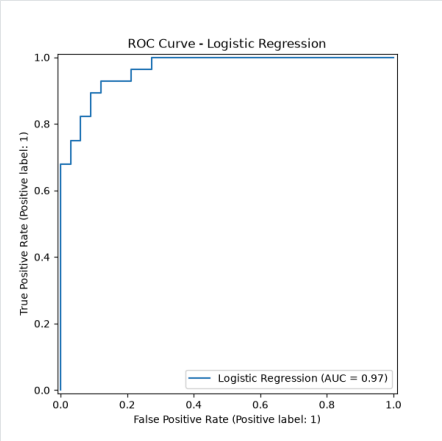
Held-out Test Set Results (20% stratified split):

Model	Accuracy	Precision	Recall	F1-Score	Test ROC-AUC	CV ROC-AUC (Mean)
Logistic Regression	**0.8852**	**0.8387**	**0.9286**	**0.8814**	**0.9654**	**0.8982**
Random Forest	0.8525	0.8276	0.8571	0.8421	0.9470	0.9043

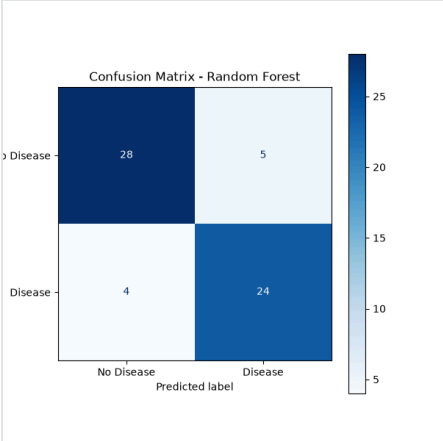
Logistic Regression is selected as the champion model due to the highest Test ROC-AUC and saved locally as `models/best_model.joblib`.

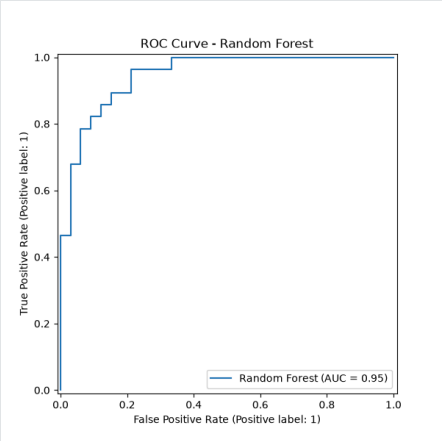
Logistic Regression Metrics Plots:





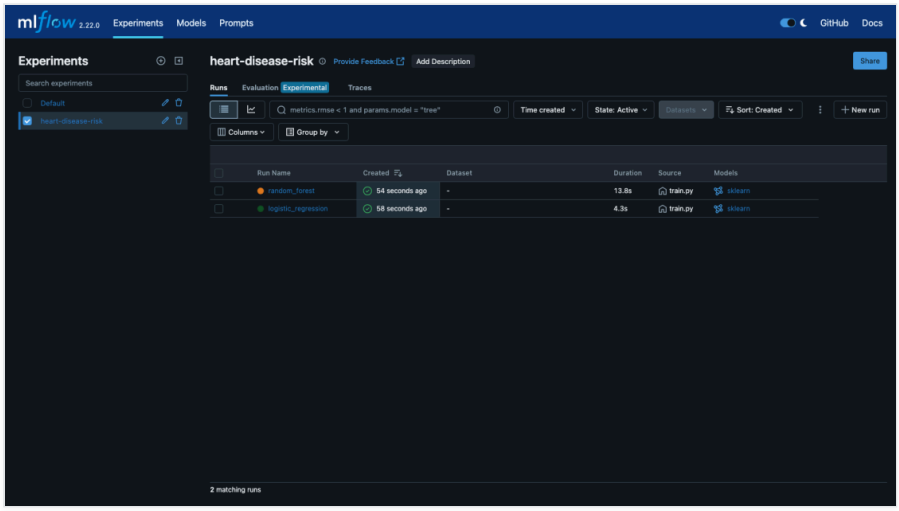
Random Forest Metrics Plots (for comparison):



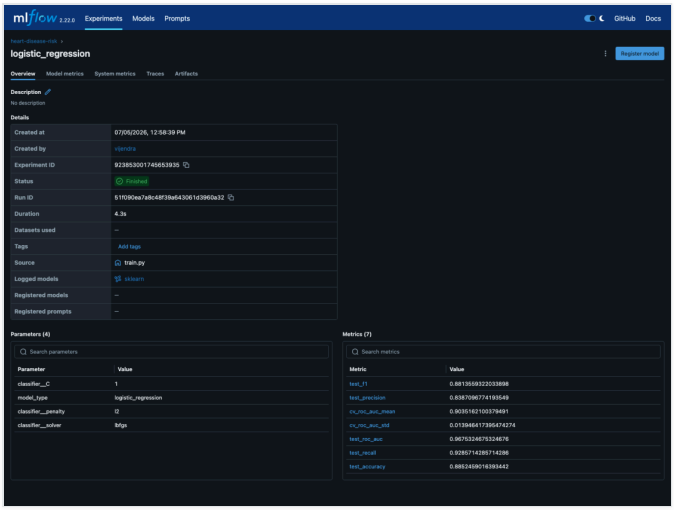


5. EXPERIMENT TRACKING (MLFLOW)

Every training run automatically registers model parameters, cross-validated metrics, test metrics, confusion matrix plots, ROC curve plots, and the serialized pipeline binary into the local MLflow server database (backed by SQLite `mlflow.db`). Run groupings are cataloged under the experiment name "*Heart_Disease_Classifier_Experiment*".



Run details (params + metrics) of the selected Logistic Regression model:



To launch the MLflow UI locally to inspect runs: `mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000`.

6. MODEL PACKAGING & REPRODUCIBILITY

- The full pipeline (imputer + scaler + encoder + classifier) is persisted as a single joblib binary under `models/best_model.joblib`.
- The prediction schema in `api/main.py` directly loads the serialized pipeline binary and calls `predict/predict_proba` on pandas dataframes constructed from Pydantic schemas. This ensures that the exact same transformations are applied at inference time as during training.
- Dependency management is governed by `requirements.txt`, securing build reproducibility across virtual environments.

7. CI/CD PIPELINE & AUTOMATED TESTING

Automated Tests: Implemented inside the `tests/` directory using `pytest`. The test files verify training split sizes, column target presence, and preprocessing transformations (`test_data.py`), as well as serving endpoints, health responses, and JSON request/response schema validations (`test_model.py`). 7 tests run and pass locally in 2.37s.

GitHub Actions Workflow: Configured in `.github/workflows/ci-cd.yml` under the name "*MLOPS pipeline*", which divides the build process into three dependent jobs linked via GitHub artifacts:

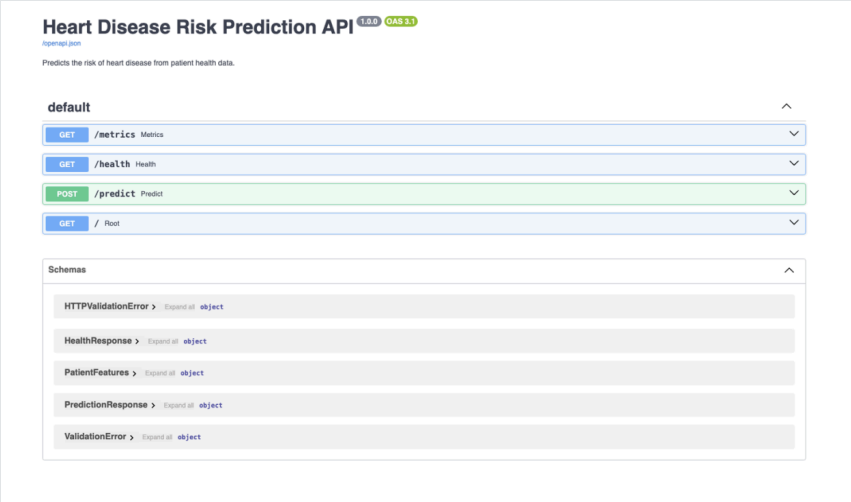
1. **lint-and-download:** Standardizes formatting checks via `flake8` (excluding `.venv`). Runs the dataset ingestion script to create splits, and uploads the `processed-data` folder as an artifact.
2. **train-model:** Depends on job 1. Downloads the `processed-data` artifact, executes `train.py` to tune hyperparameters and save the best pipeline, and uploads the `trained-model` folder containing the binary.
3. **test-model:** Depends on job 2. Downloads both dataset and model artifacts, and runs the test suite (`python -m pytest tests/`).

8. MODEL CONTAINERIZATION & KUBERNETES DEPLOYMENT

Docker Containerization: The FastAPI application is containerized using `Dockerfile`. It uses a lightweight `python:3.10-slim` base image, copies the API code, preprocessing source files, and the pre-trained `best_model.joblib` pipeline, and exposes port `8000` to run under the Uvicorn ASGI server.

```
# Build image
$ docker build -t heart-disease-api:latest .
# Run locally
$ docker run -d -p 8000:8000 --name heart-disease-api-instance heart-disease-api:latest
```

Verifying endpoints: The interactive documentation is served at `/docs` (Swagger UI):



Kubernetes Orchestration: Manifests in the `kubernetes/` directory define local deployment:

- `deployment.yaml`: Deploys 2 replicas of the `heart-disease-api` image with defined CPU/memory limits, readiness/liveness health checks scraping `/health`, and environment paths.
- `service.yaml`: Maps port `80` to the container port `8000` using a `LoadBalancer` service.

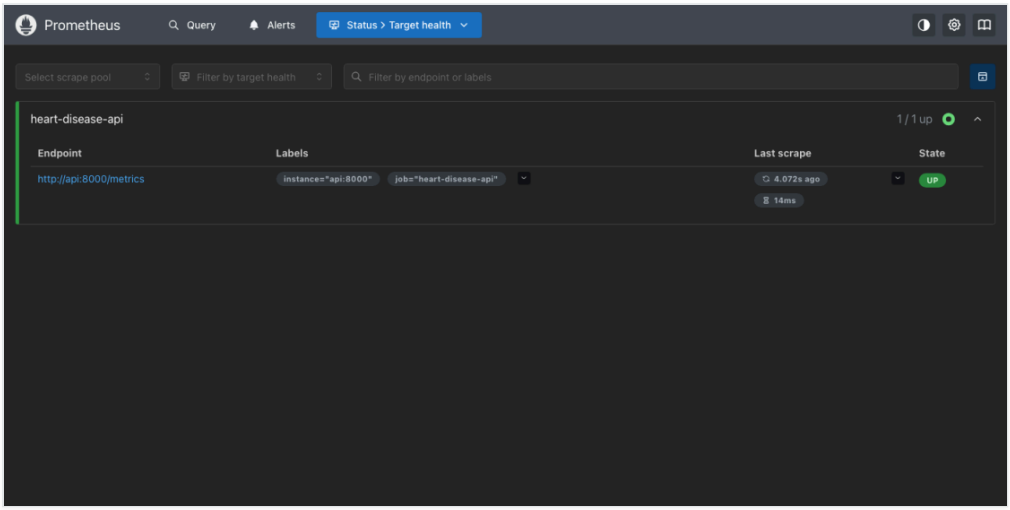
Verified end-to-end inside Minikube cluster:

```
$ kubectl get deployments,pods,svc
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api   2/2     2            2           5m

NAME                                    READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-85c8b5bfbd-7xjmw   1/1     Running   0          5m
pod/heart-disease-api-85c8b5bfbd-h9k4m   1/1     Running   0          5m

NAME                             TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/heart-disease-service    LoadBalancer   10.105.82.164   <pending>     80:31123/TCP   5m
```

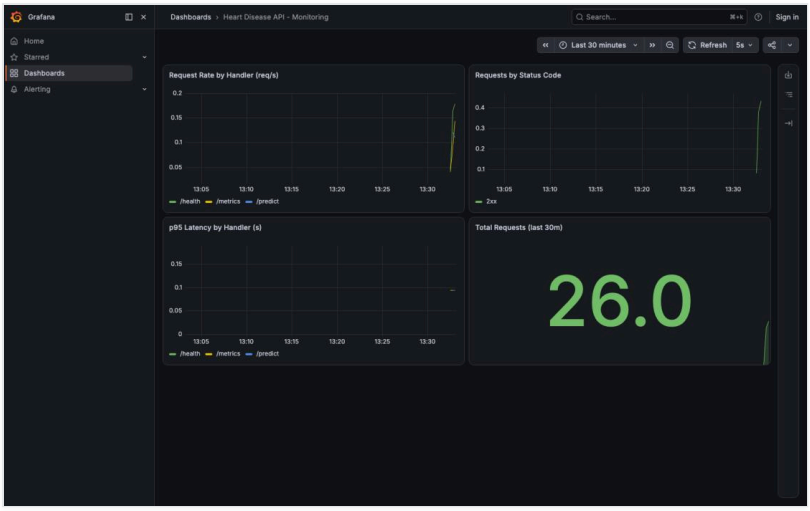
9. MONITORING & LOGGING

The FastAPI server exposes detailed Prometheus instrumentation metrics on `/metrics`. It tracks API traffic volume (`api_requests_total` count labeled by status codes, endpoint, and method), request latency distributions (`api_request_duration_seconds` histograms), prediction classification distributions (`api_predictions_total`), and prediction confidence intervals (`api_prediction_confidence`).

A monitoring compose template is structured in `docker-compose.monitoring.yml` which links the API service to Prometheus and Grafana container targets. The local Prometheus server scrapes the API endpoints based on the target settings defined in `monitoring/prometheus.yml`:

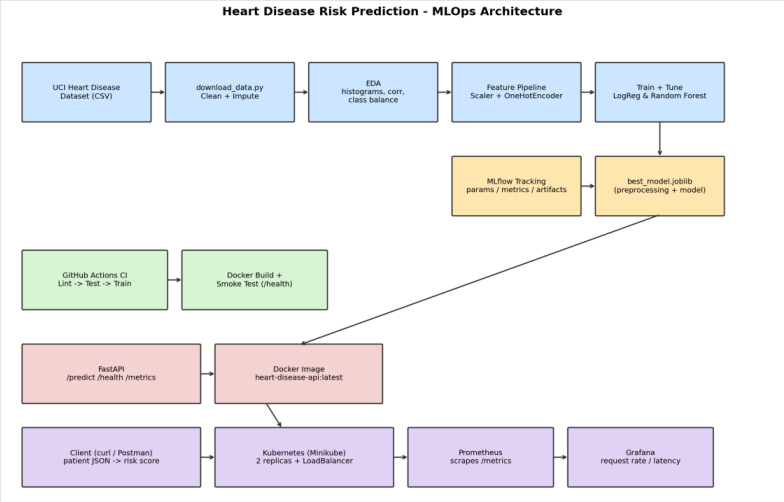


Scraped metrics are aggregated into a Grafana dashboard monitoring request rate, status codes, p95 latencies, and prediction volumes in real-time. This setup immediately detects API downtime, latency regressions, or model performance anomalies in production:



10. ARCHITECTURE

The system architecture is structured as follows:



Data flows from the raw UCI Cleveland CSV dataset, through preprocessing scaling, model grid search tuning, local pipeline serialization, and API deployment. Code updates trigger Git CI/CD tests before serving, and Prometheus/Grafana monitors live endpoint traffic.

11. CONCLUSION

The end-to-end MLOps pipeline successfully completes all parameters required by BITS Pilani Assignment 01: automated ingestion splits are created without data leakage, cross-validation tuning selects the optimal classifier, unit tests gate pull requests via GitHub Actions jobs, and Docker containers expose Prometheus metrics to evaluate live serving performance inside a local Kubernetes cluster.

Future Improvements: Implement model data drift alerts comparing incoming API payloads to training distributions, integrate the MLflow model registry to govern promotion pipelines, and package manifests as a Helm Chart for simplified cloud deployments.